Aula 07 — Expressões, Atribuição e Controle

Resumo dos 26 slides com respostas simples

Disciplina: Paradigmas de Linguagens de Programação

Slide 01 — Abertura



O que eu entendi: essa aula mostra como os valores são usados nas contas, nas atribuições e na ordem que o código roda. A ideia é tentar adivinhar o resultado primeiro e depois ver por que aconteceu.

Slide 02 — Objetivo e ligação com o mundo real

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela endentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)
Uma linha `goto fail;` duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)
No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==
`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

O que eu entendi: o objetivo é aprender como o código faz as contas e decide para onde vai. Pequenos detalhes podem dar problema de verdade, como aconteceu no caso do Zune, do goto fail e do PHP.

Slide 03 — Roteiro da aula

Roteiro da aula (80 min)

1	2	3	4	5	6
0-10 min	10-30 min	30-45 min	45-55 min	55-70 min	70-80 min
Quiz de abertura: preveja a saída	Expressões: precedência, efeitos colaterais, sobrecarga	Conversões, expressões booleanas e curto-circuito	Sentenças de atribuição	Seleção, iteração, goto e comandos protegidos	Exercício: encontre o bug

Tipo de aula: exploratória. A turma **prevê a saída antes de executar**.
Exemplos: pasta `exemplos/`, na ordem dos slides. Todos rodam no navegador em **onecompiler.com**.

Planejamento do Encontro 2 · Sebesta Caps. 7 e 8

O que eu entendi: a aula fala de expressões, conversões, if, laços e outras coisas de controle. A ideia é tentar saber o resultado, rodar e depois entender o motivo.

Slide 04 — Quiz de abertura

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula · Sebesta Cap. 7

Respostas: 1) Python: -4 e 512. 2) Java: 3345. 3) JavaScript: 3 e 52. 4) C: 0. 5) Python: “padrão” e True. 6) JavaScript: true, false, true. Basicamente muda por causa da ordem das operações e das conversões que cada linguagem faz.

Slide 05 — Expressões aritméticas: questões de projeto

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)
`(+ a (* b c))`: notação prefixa, e os parênteses dizem a ordem.

`a + b * c` com `a=3, b=4, c=5`

da esquerda para a direita
(3 + 4) * 5 = 35

da direita para a esquerda
3 + (4 * 5) = 23

com precedência (* antes de +)
3 + 4 * 5 = 23

APL: sempre da direita para a esquerda (p. 304)
3 × 4 + 5 = 27

Sebesta, 11ª ed. · §7.2 e §7.2.1 · p. 300-305

O que eu entendi: cada linguagem tem regras para saber qual conta faz primeiro. No exemplo a + b * c, a multiplicação vem antes e dá 23. Em APL a ordem é diferente e dá 27.

Slide 06 — Precedência e associatividade na prática

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
`-2 ** 2`	-4	erro de sintaxe	(não há **)
`2 ** 3 ** 2`	512	512	–
`1 + 2 + "3"`	TypeError	"33"	"33"
`-7 / 2` (inteiros)	-4 (com `//`)	-3.5	-3
`-7 % 2`	1	-1	-1

Associatividade (p. 303)
Esquerda: `* / % + -`
Direita: ** (Python, JS, Ruby, Fortran), ++, - -, menos unário
Visual Basic: ^ à esquerda!

Excel
=-2^2 dá **4**: lá o menos unário vence o ^.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. · §7.2.1.1-7.2.1.3 · p. 301-305 · exemplos/01_precedencia

Respostas: no Python, -2 ** 2 dá -4 e 2 ** 3 ** 2 dá 512. No JavaScript, -2 ** 2 sem parênteses dá erro. No Java, quando entra uma String no + ele começa a juntar o texto. Então usar parênteses ajuda a não se perder.

Slide 07 — Ordem dos operandos e efeitos colaterais

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;        /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1();   /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8 — **Java**
a ordem é garantida: da esquerda para a direita (p. 308)

20 — **C (gcc 14)**
a ordem não é definida pelo padrão

20 — **Go**
a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e `f(x) + f(x)` pode virar `2 * f(x)`.

Sebesta, 11ª ed. · §7.2.2 · p. 307-309 · exemplos/02_efeitos_colaterais

O que eu entendi: o resultado muda dependendo da ordem que as coisas são executadas. No Java dá 8, e no C e Go pode dar 20. Isso acontece porque a função muda a variável no meio da conta.

Slide 08 — Operadores sobrecarregados

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit **e** "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);        // 98
System.out.println("" + 'a' + 1);   // a1
x.add(y).multiply(tres);   // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):      # v1 + v2
        return Vetor(self.x + outro.x,
                     self.y + outro.y)

    def __mul__(self, k):          # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)      # Vetor(4, 6)
print(v * 3)                       # Vetor(12, 18)
```

Sebesta, 11ª ed. · §7.3 · p. 309-310 e §9.11 · p. 404 · exemplos/03_sobrecarga

O que eu entendi: o mesmo operador pode fazer coisas diferentes. No Java, 'a' + 1 vira 98 porque o char vira número. Já com uma String ele começa a concatenar. No Python ainda dá para mudar como alguns operadores funcionam.

Slide 09 — Conversões de tipos

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;          // alargamento implícito
System.out.println((int) f);   // 123456792 !

double d = 3.99;
System.out.println((int) d);   // 3 (cast)
System.out.println((byte) 200);   // -56

byte b = 10, c = 20;
// byte s = b + c;   // erro: b + c é int (p. 312)

System.out.println(1 / 2);     // 0
System.out.println(1 / 2.0);   // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	**não**: `float64(i)`	erro de compilação
Rust	**não**: `i as f64`	erro de compilação
ML, F# (p. 312)	**não**	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a;` com a digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. · §7.4 · p. 310-313 · exemplos/04_conversoes

O que eu entendi: converter tipos pode perder informação. Por exemplo, 123456789 vira 123456792 depois de passar por float. Go e Rust são mais chatos e obrigam a fazer a conversão manualmente em vários casos.

Slide 10 — Coerção implícita em JavaScript e PHP

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2              // 3
"5" + 2              // "52"
"5" * "2"            // 10
true + 1             // 2
[] + {}              // "[object Object]"
null + 1             // 1
undefined + 1        // NaN
"b" + "a" + +"a" + "a"   // "baNaNa"

0 == ""              // true
"" == "0"            // false
0 == "0"             // true   (não é
transitivo!)
0 === "0"            // false  (=== não
converte)
```

PHP · "type juggling"

```
var_dump("0e123" == "0e456");   // true !
var_dump("0e123" === "0e456");  // false
var_dump("1" == "01");          // true

md5("240610708")  // "0e462097431906...."
md5("QNKCDZO")    // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que **não fazem coerção**.

Sebesta, 11ª ed. · §7.4.1 Coerção em expressões · p. 311-312 · exemplos/04_conversoes

Respostas: "5" - 2 dá 3, "5" + 2 dá "52", "5" * "2" dá 10, true + 1 dá 2 e undefined + 1 dá NaN. O JavaScript faz várias conversões sozinho, por isso o == pode dar resultados estranhos. É melhor usar === e !==.

Slide 11 — Expressões relacionais e booleanas

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));   // 1

if (-1) printf("verdadeiro\n");   // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)   # True: (a > b) and (b >
c)

# contam como FALSO em um if:
# 0   0.0   ""   []   {}   None
# contam como VERDADEIRO:
# "0"   [0]   " "
```

Linguagem	O que o `if` aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	**somente booleano**: `if (n)` com n inteiro nem compila

Sebesta, 11ª ed. · §7.5 · p. 313-315 · exemplos/05_booleanas

Respostas: em C, 3 > 2 > 1 dá 0 porque primeiro 3 > 2 vira 1 e depois 1 > 1 é falso. No Python dá True porque ele entende a comparação encadeada. Algumas linguagens aceitam outros valores no if e outras só aceitam booleano.

Slide 12 — Avaliação em curto-circuito

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;          // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0   // false
s != null &  s.length() > 0   //
NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade     // undefined, sem erro

const qtd = 0;
qtd || 10              // 10  (o 0 se perdeu!)
qtd ?? 10              // 0   (só troca
null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o b++ só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve `"padrão"`: and e `or` devolvem um dos **operandos**.

Sebesta, 11ª ed. · §7.6 · p. 315-317 · exemplos/05_booleanas

O que eu entendi: o && pode nem olhar a segunda parte se já souber o resultado. Isso ajuda a evitar erro, tipo testar se é null antes de usar. No JavaScript, ?? é melhor quando 0 ainda é um valor válido.

Slide 13 — Sentenças de atribuição

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;            // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do
livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... }   // compila! atribui e
testa y
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: `if (x = y)` compila
Java, C#	sim, mas o `if` exige booleano
Python	não, exceto o operador morsa `:=`
Go	não: nem x++ é expressão
Rust	vale `()`: `if x = 5` não compila

Python · atribuição múltipla (p. 320)

```
a, b = b, a                    # troca sem
temporária
if (n := len(dados)) > 5: ...    # morsa (Python
3.8+)
```

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

O que eu entendi: em C e JavaScript dá para fazer uma atribuição no meio de uma expressão. Isso pode ser útil, mas também pode causar erro, como escrever if (x = y) sem querer. Python e Go são mais restritos nisso.

Slide 14 — Atribuição de modo misto

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;        // ok: alargamento
// int i = 3.7;      // erro: estreitamento
int i = (int) 3.7;   // 3 (cast explícito)
long l = i;          // ok

i += 3.7;            // COMPILA: cast escondido
System.out.println(i);   // 6
```

C · coerção em qualquer direção

```
int i = 3.7;          // 3, em silêncio
char c = 300;         // 44 (só um aviso)
unsigned int u = -1;  // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá **0.0**: a divisão inteira acontece **antes** do alargamento para double.

Sebesta, 11ª ed. · §7.8 · p. 321-322 · exemplos/06_atribuicao

O que eu entendi: o Java deixa algumas conversões acontecerem sozinho, mas outras precisam de cast. A pegadinha é que se os dois valores forem int, a divisão acontece como int primeiro. Por isso 7 / 10 vira 0 antes de virar double.

Slide 15 — Estruturas de controle

Estruturas de controle: quantas são necessárias?

Sequência
uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção
escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração
repetir enquanto uma condição vale

```
while (x > 0)
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O `goto` não é necessário.

Por que então tantas outras (`for`, `switch`, `do`, `foreach`)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. · §8.1 · p. 328-330

O que eu entendi: basicamente a gente precisa de código em sequência, decisões e repetições. O resto, como for, switch e foreach, existe para deixar mais fácil de escrever.

Slide 16 — Seleção de dois caminhos: else pendente

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else                    // de qual if é este else?
    result = 1;
printf("%d\n", result);   // 1
```

Regra de Java, C, C++ e C#
O `else` pertence ao `if` **mais próximo** que ainda não tem `else`. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };        // Rust: if é uma expressão (como em F#, p. 334)
y = x if x > 0 else 2 * x                     # Python: expressão condicional (p. 306)
```

Sebesta, 11ª ed. · §8.2.1 · p. 330-334 · exemplos/07_selecao

Resposta: nesse código o else vai para o if mais perto, mesmo que a identação pareça outra coisa. Por isso result termina valendo 1. Usar chaves evita essa confusão.

Slide 17 — Seleção múltipla

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
       QUI, SEX -> "dia útil";
};  // sem break, sem cair;
    // exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: `match` com padrões e guardas.

Sebesta, 11ª ed. · §8.2.2 · p. 334-341 · exemplos/07_selecao

O que eu entendi: no switch do C, se esquecer o break ele continua nos próximos cases. Então com index = 1 ele passa por vários casos e até pelo default. Outras linguagens tentam evitar isso.

Slide 18 — Laços controlados por contador

Laços controlados por contador

C · o for é um laço lógico disfarçado

```c
for (int i = 0; i < 10; i++) {
    printf("%d ", i);
    i += 2;          // altera a contagem
}                    // 0 3 6 9

for (; i <= 100; )  // expressões opcionais
    soma += i++;
```

Python · o for percorre um iterável

```python
for i in range(0, 10):
    print(i, end=" ")
    i += 2           # NÃO muda nada
                     # 0 1 2 3 4 5 6 7 8 9

for i in range(10, 0, -3):   # 10 7 4 1
    ...
```

Questões de projeto (p. 342)
Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. · §8.3.1 · p. 342-345 · exemplos/08_iteracao

Respostas: no C, mexer no i dentro do for muda a contagem, então sai 0 3 6 9. No Python, mudar o i dentro do corpo não muda o próximo valor do range, então continua 0 1 2 3 4 5 6 7 8 9.

Slide 19 — Laços lógicos, saídas e iteradores

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```java
externo:
for (int i = 0; i < m.length; i++)
    for (int j = 0; j < m[i].length; j++)
        if (m[i][j] < 0) {
            System.out.println(i + "," + j);
            break externo;   // sai dos DOIS laços
        }
```

JavaScript · for...in × for...of

```javascript
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total);   // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total);   // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```java
while (dias > 365) {
    if (bissexto(ano)) {
        if (dias > 366) { dias -= 366; ano++; }
        // dias == 366: nada muda, laço infinito!
    } else { dias -= 365; ano++; }
}
```

Pós-teste (`do-while`) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

O que eu entendi: no Zune o while entrou em loop infinito porque chegou numa situação em que nada mudava. No JavaScript, for...in pega os índices e for...of pega os valores. Confundir os dois pode dar resultado errado.

Slide 20 — Desvio incondicional: goto

Desvio incondicional: o goto

"goto fail" (Apple, 2014) · versão simplificada

```
int valida_certificado(void) {
    int err;
    if ((err = verifica_hash()) != 0)
        goto fail;
        goto fail;          // duplicada: desvia
SEMPRE
    if ((err = verifica_assinatura()) != 0)
        goto fail;          // nunca é alcançada
fail:
    return err;             // 0 = "certificado
válido"
}
```

Dijkstra (1968)
"Go To Statement Considered Harmful"

Quem tem goto hoje
C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos
`break`, `continue` e `return` melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com `gcc -Wall` (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. · §8.4 · p. 352-353 · exemplos/09_desvio_protegido

O que eu entendi: no caso do goto fail tinha uma linha repetida que parecia estar dentro do if, mas não estava. Aí o programa pulava uma verificação importante. Usar chaves e prestar atenção nos avisos do compilador ajuda.

Slide 21 — Comandos protegidos de Dijkstra

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se x == y, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no `if`) ou fim do laço (no `do ... od`)
- Base do **CSP** de Hoare (1978) e do `select` de Go

Go · select, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495]  (muda a cada
execução)
```

Sebesta, 11ª ed. · §8.5 · p. 353-355 · exemplos/09_desvio_protegido

O que eu entendi: nesses comandos tem várias condições. Se mais de uma funcionar, uma delas pode ser escolhida. O select do Go usa uma ideia parecida quando mais de um canal está pronto.

Slide 22 — Exercício em duplas: encontre o bug (1/3)

Exercício em duplas: encontre o bug (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

1 · JavaScript

```
const precos = [10, 20, 30];
let total = 0;
for (const p in precos)
    total += p;
console.log("Total: " + total);
```

2 · Python

```
def desconto(preco, d=None):
    d = d or 10     # padrão: 10%
    return preco * (100 - d) / 100

print(desconto(200, 0))
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Questão 1: sai "Total: 0012" porque for...in pega os índices como texto. O certo é usar for...of ou reduce. Questão 2: sai 180.0 porque o Python considera 0 como falso e troca por 10. O certo é testar se d is None.

Slide 23 — Exercício em duplas: encontre o bug (2/3)

Exercício: encontre o bug (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

3 · Java

```
int dia = 2; String nome = "";
switch (dia) {
  case 1: nome = "domingo";
  case 2: nome = "segunda";
  case 3: nome = "terça";
  default: nome = "inválido";
}
System.out.println(nome);
```

4 · C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Questão 3: imprime "inválido" porque faltou break e o switch continua até o default. Questão 4: imprime "saldo insuficiente" e "fim" porque tem um ; logo depois do if. É só tirar esse ; e usar chaves.

Slide 24 — Exercício em duplas: encontre o bug (3/3)

Exercício: encontre o bug (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual regra causa o problema? **(c)** como corrigir?

5 · Go

```go
x := 10
if x > 5 {
    x := x * 2
    fmt.Println("dentro:", x)
}
fmt.Println("fora:", x)
```

6 · Java

```java
int acertos = 7, total = 10;
double taxa = acertos / total * 100;
System.out.println(taxa + "%");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) · execute em onecompiler.com

Questão 5: imprime "dentro: 20" e "fora: 10" porque := cria outro x dentro do if. Tem que usar x = x * 2. Questão 6: imprime "0.0%" porque 7 / 10 é feito como inteiro e vira 0. Tem que colocar double na conta.

Slide 25 — Síntese: armadilhas e como as linguagens ajudam

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	`-2 ** 2`, `1 + 2 + "3"`	parênteses; JS obriga `(-2) ** 2`
Operandos	`a + f()` com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; `===`
Booleanos	`0`, `""`, `[]` são falsos; `a > b > c` em C	`if` só com booleano (Java, Go, Rust); `??`
Atribuição	`if (x = y)`	atribuição não é expressão (Python, Go, Rust)
Seleção	`switch` sem `break`; else pendente	`->`, `match` exaustivo, chaves obrigatórias
Laços	`for...in` no JS; laço sem progresso	`for...of`, iteradores, `break` rotulado
Desvio	`goto` sem restrição	`break`, `continue`, `return`: gotos restritos

Sebesta, 11ª ed. · Caps. 7 e 8 · p. 299-361

O que eu entendi: a aula mostrou várias pegadinhas de linguagem, tipo ordem das contas, conversão automática, if, switch e laços. Linguagens mais novas tentam bloquear alguns desses erros.

Slide 26 — Atividades complementares

Atividades complementares (autopesquisa)

- **Expressões em Ruby e em Lisp** (§7.2.1.4-7.2.1.5, p. 305-306): `expressoes.rb`, `expressoes.lisp`
- **Expressões condicionais** (§7.2.1.6, p. 306) e **erros em expressões** (§7.4.3, p. 313)
- **Implementação da seleção múltipla**: tabelas de desvio (§8.2.2.3, p. 337-339)
- **Laços em linguagens funcionais**: contagem por recursão (§8.3.1.4, p. 345)
- **Iteradores e blocos de Ruby** (§8.3.4, p. 349-352)
- Leitura: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com `switch` em C, `switch ->` em Java, `switch` em Go e `match` em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)

Sebesta, 11ª ed. · Caps. 7 e 8 · exemplos/11_autopesquisa

O que eu entendi: no final tem alguns assuntos para pesquisar mais, como Ruby, Lisp, switch e match. A atividade é comparar como C, Java, Go e Rust fazem seleção múltipla.